

A3 · Interrogating the Transport Layer

CS425 Computer Networks · in class activity · I drive Claude Code, then your group drives

TURN THIS IN ON PAPER

before you leave the room
graded pass/fail

Names _____ Date _____

How every round works

Predict before anything runs; a prediction written afterwards is not one. **Prompt:** read each action Claude asks permission for. **Run** and write what you see. **Check:** were you right, was Claude right, and which section of chapter 3 says so? Wrong predictions cost nothing.

ROUND 0 · MEET THE AGENT

```
mkdir a3-demo && cd a3-demo && claude
```

1. Claude asked whether you trust the files in this folder. Why is "yes" safe here and dangerous in your home directory?

2. What is the difference between approving one action and telling it not to ask again?

ROUND 1 · DEMULTIPLEXING · §3.2

Write a Python 3 program `echo.py`, standard library only, that runs a TCP echo server and a UDP echo server at the same time, both on 127.0.0.1 at a port given on the command line. For every TCP connection and every UDP datagram, print the client's IP address and port.

```
python3 echo.py 5425
nc 127.0.0.1 5425          # two terminals, left open
echo hi | nc -u -w1 127.0.0.1 5425  # twice
lsof -nP -i :5425
```

PREDICT	Can TCP and UDP both bind port 5425? With two TCP and two UDP clients, how many sockets does the server hold?
OBSERVE	Server's lines in <code>lsof</code> : TCP LISTEN _____ TCP connected _____ UDP _____
CHECK	Which lines show a four-tuple, and which a two-tuple? Why does a web server need the four-tuple?

ROUND 2 · WHO IS LEFT HOLDING TIME_WAIT · §3.5.6

I am about to close one of the TCP clients connected to echo.py. Which side of that connection will end up in TIME_WAIT, for how long, and why does TCP need that state at all?

```
netstat -an -p tcp | grep 5425          # macOS; Linux: ss -tan | grep 5425
```

PREDICT	Client or server in TIME_WAIT? For roughly how long?
CLAUDE SAID	
OBSERVE	Whose port was in TIME_WAIT? Then the server was stopped first and restarted: did the restart work?
CHECK	Is SO_REUSEADDR in echo.py? Did Claude's explanation of it match what happened?

ROUND 3 · A BYTE STREAM IS NOT A MESSAGE · §3.3, §3.5.2

Change the TCP side of echo.py to wait one second after accepting a connection before its first recv. For both TCP and UDP, print every recv on its own line with its length and the bytes received. Then write burst.py, which sends "one", "two" and "three" as three separate sends with no delay, first over TCP and then as three UDP datagrams to the same port.

```
python3 echo.py 5425
python3 burst.py 5425
```

PREDICT	recv calls that return data: TCP _____ UDP _____
OBSERVE	What did the server print for each, with lengths?
CHECK	Explain the TCP result using the phrase "byte stream". Why did your P1 client have to look for <code>\r\n</code> rather than trust one recv to be one reply?

ROUND 4 · YOUR TURN

One laptop with a command line coding agent (Claude Code, Copilot CLI, Codex CLI, or similar), in a new empty directory. No agent in the group? Do it by hand; I will put Claude's answers on the projector at the end.

Agent we used _____ (or "by hand")

Part A · Repeat a step, in your own words

ROUND REPEATED	SAME CODE AS THE PROJECTOR? SAME BEHAVIOR? IF NOT CLAUDE CODE: HOW DID YOUR AGENT'S CODE, PERMISSION REQUESTS AND EXPLANATIONS DIFFER?

Part B · Check its arithmetic

QUESTION	YOUR AGENT'S ANSWER	YOUR ANSWER, BY HAND
UDP checksum. Words 01010011, 01100110, 01110100. One's complement sum, and the checksum?		
Stop-and-wait. 1 Gbps, 30 ms RTT, 1000 byte packets. Utilization? Packets in flight for 90%?		
Timeout. EstimatedRTT 100 ms, DevRTT 5 ms, next SampleRTT 120 ms, $\alpha = 0.125$, $\beta = 0.25$. New TimeoutInterval? Which EstimatedRTT went into DevRTT, old or new?		

1. Where did the agent and your group disagree, and who turned out to be right? If you always agreed, what did you check, and how?
-
2. Name one thing today where you would not have trusted an agent's answer without running it.
-

If you finish early: ask your agent why TIME_WAIT lasts about 30 s on macOS and 60 s on Linux, then run `sysctl net.inet.tcp.msl` on a Mac. What is MSL? _____

Graded pass/fail on a serious attempt at every round. Hand this in before you leave, with every name at the top.